

Merge K Sorted LL

k elements/lists.

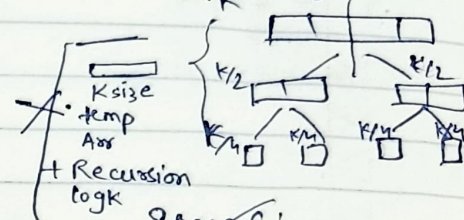
Using heap

merge first +
then not
2 & soon

- ① $n^2 K, S.C: O(1)$
- ② Mergesort

T.C $\Rightarrow nK \log K$

S.C: ~~$nK \log K$~~



Space C:

it is
LL not
Arr,

$O(K) + O(\log K)$
 $\approx O(K)$

merge func
will not consume
any extra space.

S.C: $\log K$
due to
Recursion.

Approach 1: using heap. (minheap)
size nk

all nk nodes into
the heap. & step down Approach

+ all nk nodes * $\log nk$ for
retrieval.

T.C: $O(nK + nk \log nk)$
S.C: $O(nK)$

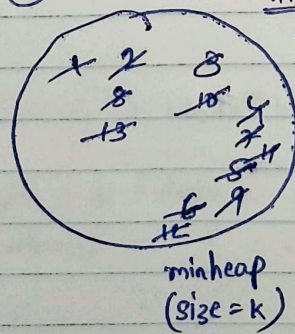
Approach 2:

- ① first K elements into heap
either K or $K \log K$ T.C
- ② Total elements: nk

Among which k is
already inserted,

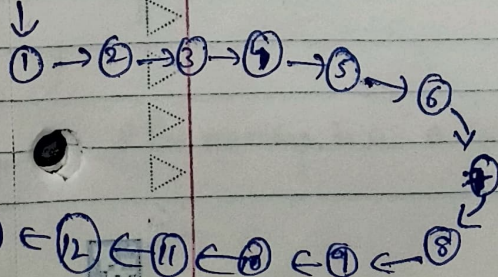
$(nk - k) * 2 \log K$
 $K \log K + nk \log K - K \log K$

T.C: $O(nK \log K)$
S.C: $O(K)$



minheap
(size = k)

head



In this Approach 2,

Now here we cannot simply
store values, we need to
store addr of Node in the heap,
only then will be able to
fetch details of next Node in
the list.

But, then we also
have to specify that
minheap should perform
operations based
on data/values of the Node
& not address,

Let's see how we can do
that

How we will store addresses of the Node in the minHeap but operations will be performed based on data/values?

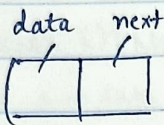
Class Node {

public:
int data;
Node* next;

Node(int x){

data = x;
next = NULL;

}



How normally we would've created a minheap?

priority_queue<Node*, vector<Node*>, greater<Node*>> p;

⇒ How are we going to do here?

priority_queue<Node*, vector<Node*>, Compare> p;

Class Compare {

public:

bool operator()(Node*a, Node*b)

{

return a->data > b->data;
// for minheap

}

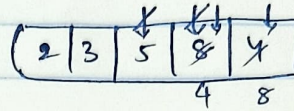
}

// for maxheap, it is a->data < b->data;

Normal Scenarios

bool operator (inta, int b){

return a <= b; // for increasing order
}



a = 5 b = 8

5 <= 8 ✓

Condition true,
it will not swap

8 <= 4

a <= b ✗

false

it will swap.

& for decreasing order

return a >= b;

But in case of heap, it is opposite

a->data > b->data
(minheap)

& a->data < b->data
(maxheap).

Similar to providing
a Comparator to collections.sort()
to define custom sorting logic

Merge K Sorted LL :

Code :

```
class Compare {  
    public:  
    bool operator () (Node* a, Node* b)  
    { return a->data > b->data;  
    }  
};
```

```
Node* mergeKLists (Node* arr[], int K)  
{
```

```
    // Minheap
```

```
    priority_queue<Node*, vector<Node*>, Compare> p (arr, arr+K);
```

```
    Node* root = new Node(0);
```

```
    Node* tail = root;
```

```
    Node* temp;
```

```
    while (!p.empty()) {
```

```
        temp = p.top();
```

```
        p.pop();
```

```
        tail->next = temp;
```

```
        tail = tail->next;
```

```
        if (temp->next)
```

```
            p.push(temp->next);
```

```
    }
```

```
    return root->next;
```

Entire code will be

same & simple

Only thing we need

to ensure is

operations being performed

based on data/values